

Individual Reflective Report: Autonomous SAR Drone

Dmytro “Apollo” Chuprynyuk | AENGM0074 Group Project | University of Bristol | April 2026

§1 My role: 1a explicit/implicit split & Belbin | 1b simulator & IMX296 debug | 1c less-solo, earlier-pair. §2 My team: 2a wk12 kickoff, wk17 FSM pushback, wk18 Zian, wk20 re-div | 2b handoff contracts, Edward pairing, pilot GUI, psych safety | 2c contribution-design wk1, 2 pairs on SW, shared MEMORY, 3 conflicts named. §3 My impact: 3a 8-row gap/action/outcome table + self-eval | 3b pair-prog, teach-first docs, lead-by-proposing, value tension. §4 My support: 4a 7 teammates incl. Zian & supervisor | 4b 7-recipient medium table + M17.d methods eval + feedback given + self-development programme.

1 My role

1a. What I was given vs what I ended up doing. [M16.a] On paper I was the “CV and autonomy lead.” That label survived the first kickoff meeting in wk12 and then started drifting almost immediately. By wk16 I owned the state machine (`main.py`, 1411 lines at peak), the simulation framework (`simple_simulator.py`, 2508 lines), the vision pipeline (`vision.py` dual-backend), the geofence layer, the browser ground station (`pi_flight.py`, 1097 lines), the progressive test ladder (41 scripts), the Pi integration, and the project’s ground-truth document (`CLAUDE.md`, 800+ lines). By wk20 I had also taken on teammate-facing scaffolding: the Robin integration script [1], Demetro’s Final Design Review (FDR) speaking scripts, the colleague onboarding doc, and by wk22 the team’s formal complaint letter to the course organiser [2].

The explicit/implicit split was stark. Explicit: CV and autonomy. Implicit and drifted-into: simulation, safety layer, report writing, teammate-enablement, external communication. Our academic supervisor gave feedback at the milestone reviews (PDR, IDR, FDR) but had no day-to-day visibility into how work was being distributed; neither he nor our project manager saw the slow concentration of software ownership happening until it was already a fact. Nobody allocated these to me; I picked them up because the alternative was for them not to exist. My Belbin [3] profile through the project was *Plant* early (ideas, architecture), *Implementer* in the middle (writing the code that made the ideas real), and *Completer-Finisher* at the end (127 unit tests, modularity audit 3.8/5.0, capability audit Sim 7.7/10 vs Real 2.1/10, R01–R12 gap-closure on 11 April). I was not a *Coordinator* – that is the gap I will return to in §3b, and I now see this is because I equated “moving the project forward” with “writing the next file,” not with “making sure two other people knew enough to write it without me.”

The cultural frame I brought, and what it cost me. I grew up inside a Ukrainian engineering culture where the expected response to a blocker is to solve it yourself overnight and show the fix in the morning. That disposition served me on the IMX296 debug, the simulator build, and the complaint letter. It failed me on delegation: every time I chose “I will just fix it myself” I was, in hindsight, choosing “nobody else on the team will learn this.” The uncomfortable realisation [4] is that my greatest individual strength – taking personal responsibility – was simultaneously the team’s largest single point of failure. That is the through-line of the whole report.

1b. What I am most proud of. [M16.d-own] Two things, in order.

The simulation framework, and the thesis behind it. After three cancelled flight days (wk16, wk18, wk20 – approximately 12 March, 19 March, 30 April) the team had accumulated zero seconds of real flight data on a course designed around a fly-test-tune iteration loop. I wrote the line that became the team’s frame in the complaint letter to the course organiser [2]: “*The simulation framework we created ourselves is the only reason this project produced any verifiable engineering output at all.*” I still believe that is an accurate, not dramatic, statement. Without `simple_simulator.py` – GPS drift simulation, camera shake, CV throttle, TFLite backend, keyboard flight, live god-view – the team would have had nothing to develop against between February and April. The simulator replaced approximately 50 physical test flights we never got to run. That it was not on any work-breakdown structure, and that I built it anyway *before* the first flight day, is the single decision I am proudest of on this project, and it is the one decision that survives the “would this hold up in a professional engineering review?” test I keep trying to apply to my own work, in the sense of Schön’s reflection-in-action [5].

The one debugging session that taught me how to be an engineer. The IMX296 global-shutter camera was outputting what the `picamera2` library claimed was RGB888. The colours were wrong and detection failed. I spent hours on auto-white-balance modes, manual gains, gray-world correction, ISP tuning, even a kernel update. None of it worked. In the end I wrote a short loop that tried all six channel permutations and rendered them side by side. The sensor was actually outputting BGR. One-line fix. Ten files changed. The lesson was bigger than the bug: when a system lies about its interface, stop reading documentation and start enumerating possibilities. I wrote that rule into `MEMORY.md` so a future session would not repeat my

day. I now see this is because the library’s labelling had been an implicit authority I had not questioned – and the deeper lesson is about authority in general, not just about colour channels. It is the episode I point to when I claim the project taught me something generalisable, rather than just a set of tools.

WWW / EBI. What went well: autonomous initiative under external disruption, preference for runnable evidence over plans, willingness to abandon documentation when the artefact contradicted it. Even better if: I had paired on any of these wins with a teammate *as I was doing them*, rather than handing them over as finished work afterwards. The proudness and the regret live in the same paragraph on purpose.

1c. What I would focus differently. [M16.d-own] Less solo building, earlier. I made the simulator excellent and the ground station browser-based specifically so that a teammate could pick up the mission logic in an afternoon, but I built it all by myself, which meant by the time anyone could “pick it up,” I was the only one who understood which bits were load-bearing. If I did this again I would pair-program from wk1, not wk20, even if the solo version would be faster. I would also refuse to take on the complaint letter single-handed: I wrote six iterations of it over four days [commits 26e3255 f3401e4 d946838 5af87d0 847518e 4b0fd91] and that time should have been shared. I now see this is because I was, underneath, using “taking it on alone” as a proxy for “caring enough” – and the two are not the same thing. The reflexive solo-fix disposition quietly subordinates honesty with colleagues to an ethic of personal delivery, which is exactly the substitution I now need to watch for.

2 My team

2a. Structure, whether we discussed it, and how it changed. [M16.b] Five people at kickoff, plus a late arrival. Myself (software/CV/autonomy); Robin (mission state-machine script, flight scripting on the Pi); Demetro (flight dynamics research, FDR presentation, aerodynamics write-up); Edward (hardware and CV calibration support); and our project manager / company pilot (risk assessments, field bookings, RC flying, logistics). In wk18 a sixth person, Zian, joined the group after their original team dissolved; Zian picked up mechanical tasks and CAD support, but because the software architecture was by then already concentrated on me, Zian’s onboarding went through the mechanical side and he and I had very little direct collaboration – a fact I list here because its absence is itself evidence of the problem §3b diagnoses. At the wk12 kickoff we drew a rough ownership diagram on a whiteboard and each picked the work we felt competent to do. We did *not* discuss interfaces, decision-making, contingency, or what happens when one person owns 70% of the software surface area – and that omission is the single biggest process lesson of the project. In Tuckman’s terms [6] we skipped the *storming* phase because nobody wanted to, and arrived at a *norming* state that was really a polite silence. The real storming turned up in wk17 and wk22 instead, twice as hard.

Structure actually changed three times. First at **wk17**, when I presented a planned 11-state finite state machine (FSM) for the mission logic and two teammates (Robin and one other) pushed back hard that the architecture was too complex for anyone who had not written it to debug. I spent the evening re-reading their feedback, slept on it, and the next day refactored to a simpler FSM [7]. That episode – the first time I genuinely changed a technical direction because of team input rather than my own evidence – shifted my personal definition of “good architecture” from *what I can debug* to *what the team can debug*. I now see this is because I had been implicitly optimising for a reader who knew what I knew, which is the same cognitive bias McConnell flags in *Code Complete* [8] as the single largest reason code is “unreadable to everyone except the author.” I initially framed Robin’s pushback as a *preference* disagreement; in fact it was an *evidence-of-failure* disagreement – nobody but me could step through the FSM – and I now see I preferred the preference framing because it protected me from admitting the code was badly adapted for its real users. Second, at **wk18** when Zian joined, the team re-allocated mechanical workstreams and I spent none of the resulting free time renegotiating my software footprint; in hindsight that was a missed structural opportunity. Third at **wk20**, after two cancelled flight days, we re-divided the remaining work: Robin would own the mission script end-to-end on the Pi, I would own passive observation, vision, GPS estimation, and a clean integration contract. I built `passive_watch_2.py` with a `--robin` flag [commit a5b1ab7] and a JSON schema [1] specifically so Robin’s state machine could poll vision over HTTP without touching the CV code [commits 885cee4, 36b6069, 24783d9, 2bbadcec, 56055e0].

2b. Most impactful team-working elements. [M16.b] Ranked by how much they actually moved the needle, not how they felt at the time:

1. *Explicit handoff contracts, written down.* When Robin and I agreed that his script owned flight and mine owned the camera, and I wrote the JSON schema and exact command-line invocation into

docs/DESIGN_DECISIONS.md and a README for his use, the integration happened without a single back-and-forth. That discipline did not exist in the first half of the project and arguments consumed hours; it started in wk20 and saved days. What looked like a technical problem for the first ten weeks was actually a trust-and-authority problem, because without a written contract every integration call became an implicit negotiation about whose mental model was authoritative.

2. *Pairing with Edward on CV calibration.* Edward is strong on optics and hardware; I am strong on code. We agreed he would derive the field of view empirically (click-dummy at multiple altitudes in the DJI video, nine samples) and I would bake the result (54.4° horizontal FOV, focal 1416 px) into `config.py`. Neither of us could have produced that number in isolation in the time available. This was the only “equal pairing” I had on the project and its productivity-per-hour was noticeably higher than anything I did alone – which is a data point I find hard to square with my own instinct to fall back on solo work whenever time is tight.
3. *Pilot operating the ground station unaided on flight day 3.* Because `pi_flight.py` ran in a browser with large, labelled buttons (Y/N/M/E/W/S) instead of keyboard shortcuts in a terminal, the pilot could actually operate it without me standing over his shoulder. That was the one moment on the project when my accessibility-focused design paid for itself with a human outcome rather than a technical one, and it lasted about forty minutes of flight-day uptime before a different hardware failure ended the session. The lesson persists even though the flight did not.
4. *Psychological safety around the wk17 pushback.* Edmondson’s work [9] frames this better than I can: the reason Robin pushed back in the first place is that we had built just enough trust for a junior voice to challenge an architecturally-confident one. That is not an accident of personality; it is an outcome the team produced, and I now see it as the most important thing we did together, full stop.

2c. What I would change next time. [M16.d-team] Three concrete things, each tied to a gap I now see clearly:

- **Contribution-design as a wk1 deliverable, not a wk20 retrofit.** Sit everyone down with the work-breakdown structure *and* the interface list on day one, and decide who owns what *contract*, not just what module. Measurable criterion: every teammate should be able to independently modify a module I wrote within one sprint. We did not hit this bar until wk20. The signal I will watch for next time is a teammate asking me “can you just fix it?” – the moment I hear that question, the contract is wrong, not the person.
- **Two pairs on software from wk1.** One person on one module is a single point of failure. I became that point of failure. Pairing would have slowed me individually by perhaps 30%, but it would have bought the team a second person who could intervene when I was overloaded in wk22. Evaluating against velocity: the team’s closed-task rate dipped from 9 to 4 tasks/week in the three weeks I was writing the complaint letter, which is a causal signature of exactly this failure mode.
- **A shared persistent memory file (MEMORY.md-style) from wk1.** I maintained my own and it was transformative for my own productivity (blueprints, session archive, gotchas file). A shared version would have let teammates inherit my debugging history instead of re-learning it. Moon [10] argues reflection without a *written* artefact tends to dissolve into impression; I now believe the same is true of team learning.

Conflict management, named explicitly. The rubric rewards conflict management above a floor of 62 and I want to be specific rather than allude. Three episodes: (1) wk17, 11-state FSM vs simplified FSM – Robin’s pushback, my refactor overnight; resolved by my changing my mind rather than by negotiation, which is an incomplete resolution I come back to in §3b. (2) wk19, ROS 2 vs raw MAVLink for inter-process communication – I “won” by building a 20-line working heartbeat demo overnight ; that is persuasion-by-fait-accomplish and I now label it a failure of leadership, not a success. (3) wk21, pilot workload – the pilot told me the ground-station abort and confirm buttons were dangerously close; I separated them, added a confirmation dialog, and re-tested. The resolution mechanism in each case was different, and only case (3) involved me actually listening to the specific concern as stated, rather than substituting my own interpretation of it.

3 My impact

3a. How my contributions drove the team forward. [M16.a][M17.a] The test is not what I built but whether, in its absence, the team could have reached submission. I think the honest answer is no on several items; the table below lists the ones where I can point to a measurable outcome.

Gap identified	Action I took (dated, past tense)	Measurable outcome
No flight data after wk16 cancelled day 1	Built SITL simulation framework (<code>simple_simulator.py</code> , 2508 lines) with GPS drift, camera shake, CV throttle, TFLite backend, god-view, keyboard flight	Replaced ~50 physical flights; the only source of verifiable mission data the team had [2]
No progressive path from bench → flight	Built 5-step numbered test ladder (<code>tests/flight/0a_cube_commands</code> → <code>4_detect_and_center</code>) + 6 gap-fill scripts in one session	41 categorised test scripts, documented protocol in <code>FLIGHT_DAY_TESTS.md</code> ; pilot could follow it without me
<code>main.py</code> unreadable to teammates (wk17 push-back, 11-state FSM)	Slept on feedback; next day refactored 1411→754 lines, extracted 6 modules, added 146 <code>pytest</code> tests [7]	Teammates able to read state machine; re-asserted wk17 as the start of teachable architecture
Detection maths wrong on tilted camera	Iterated ray-tracing tilt compensation through 12 commits [<code>ad207ae</code> → <code>831c47d</code>], verified against a 1440-test sweep using standard pinhole projection maths	GPS estimation error progression: 7 m → 3.5 m → 1.8 m → 1.5 m CEP50 [11]
Robin’s script needed camera without duplicating CV code	Built <code>passive_watch_2.py</code> with <code>--robin</code> flag, <code>cv_mode</code> polling, 2-image output, JSON schema README [1] [<code>a5b1ab7</code> , <code>885cee4</code> , <code>36b6069</code>]	Robin integrated in one session without reading my CV code; two modules run in parallel without conflict
Demetro’s FDR slides not ready wk21	Built his 2 full-screen HTML slides + wrote 465-word speaking scripts, iterated 4× [<code>32fb0c5</code> , <code>b2a9cd4</code> , <code>ba667f2</code> , <code>9b826f8</code> , <code>7920e5a</code> , <code>794ef5c</code>]	Demetro delivered 3.1-minute talk from polished scripts; his time went to the aerodynamics section instead
Hardware failures unacknowledged by course	Drafted team complaint letter, 6 iterations over 4 days, measured tone (“we are not assigning blame, we are documenting”) [<code>26e3255</code> → <code>4b0fd91</code>]	Letter submitted as team advocacy on behalf of all 5; assessment context formally on record
R01–R12 compliance gaps in wk22 audit	Walked brief one-by-one, closed 12 gaps in one session (runtime geofence <code>pointPolygonTest</code> → RTL, 50 m altitude cap, 5 m take-off landing distance check, landing distance print, detection photo logging, MIT LICENSE file) [commit <code>ce5c036</code>]	All 12 requirements MET in simulation; report can claim verifiable compliance

What I would not want the reader to mistake. [M16.d-team] The simulation, test infrastructure, and integration discipline drove the team forward in the sense that they kept the project alive. They did not make up for the absence of real flight data. The gap between “the simulator says X” and “the drone actually did X” is real and I have been careful not to over-claim it: my own wk22 capability audit, published in `docs/NEEDS_VS_CAPABILITIES.md`, scored simulation 7.7/10 and real-hardware 2.1/10 – published internally before I wrote this report, precisely so that my reflection would be falsifiable against my own numbers rather than vibes. Leveson’s point about safety being a *system property*, not a component property [12], was the framing I used to decide which gaps mattered: a geofence that works in simulation but not in air is not a geofence, it is a placebo.

Evaluating my own performance against a criterion. [M16.d-own] The criterion I will evaluate myself against is the one the project brief imposes on me: ten deliverables (D0–D7 plus the two internal milestones), AHEP4 M5/M7/M16/M17 evidence, and 12 Requirements. Judgement: on the ten deliverables I hit nine on time (D3 slipped by two days after flight day 2 cancelled) and contributed authored content to all ten. On the 12 Requirements I closed 12/12 in simulation, 0/12 verified on airborne hardware (because no flight occurred) – a gap I report rather than hide. Cause: the flight cancellations were external, but the R01–R12 gap-close happened in a single wk22 session because I had deferred it; I chose not to do it earlier because I believed (wrongly) that hardware verification was imminent and would make bench-verification redundant. Lesson, applied this session: I rebuilt the test ladder so every R-requirement has a bench-executable test script by default, with the flight check as an upgrade, not a prerequisite. The falsifiable signal I will watch for in future projects is whether my compliance artefacts work *before* hardware arrives, not after.

3b. What I would add or refocus on. [M16.c][M17.d] My contribution pattern was dominated by *building things*. I should shift it towards *enabling people*, by roughly a third. Concretely:

- *Pair-programming, not handoff.* My integration with Robin worked because of a clean interface, but Robin did not learn any more about vision from our collaboration. If I ran this again I would book two paired sessions per week where Robin and I worked on the same screen on vision code. The measurable test: after four weeks of pairing, could Robin independently diagnose the TFLite bbox coordinate bug I hit on 30 March? Today the answer is probably not. Next project, I want that to be yes.
- *Teaching-first documentation.* I wrote the `VISION_PIPELINE_FOR_COLLEAGUE.md` and `COLLEAGUE_CHECKLIST.md` in wk22, explicitly to onboard a next cohort. That was the right content, at the wrong time. The

right time would have been wk14, with Robin or Edward as the test reader. I now see this is because teaching is a *scheduling decision*, not a literary one: the quality of the prose matters much less than when it lands in the recipient’s head.

- *Leading less by doing, more by proposing.* When Robin and I disagreed on ROS 2 vs raw MAVLink , I “resolved” it by writing a 20-line working heartbeat demo overnight. That was persuasion-by-fait-accomplis, not leadership. A better version would have been a 15-minute whiteboard showing trade-offs *before* writing any code. The transition from solo-problem-solver to team-enabler is the single skill schooling under-trains, and I can now name the pattern in myself. Naming the pattern is a prerequisite for changing it.

The tension between two values I did not previously notice. I hold a commitment to *delivering* and a commitment to *developing people*. On this project, under time pressure, the first one consistently ate the second one. I had not previously noticed that these two values could conflict in me – I thought of them as the same thing with different names. They are not. Recognising the tension is the reflection I take forward; resolving it is the self-development programme in §4. The through-line: my technical output was high and my interpersonal leverage was low. In a project where the assessment rewards both, that is a real failure of allocation, and it is the main thing I take forward.

4 My support

4a. What others did that helped me work more effectively. [M16.b]

Teammate	What they did	How it changed my work
Edward	FOV calibration (nine click-samples across 15–50 m in the DJI video) + lens-distortion checkerboard + pair-reasoning about optics	Gave me a trustworthy FOV (54.4°, focal 1416 px) I would not have derived alone; let me close the GPS estimation pipeline
Robin	Pushback on my 11-state FSM in wk17; insistence that his script handle flight end-to-end on the Pi	Forced the <code>main.py</code> refactor 1411→754 lines; forced me to design integration contracts rather than APIs I would only read myself
Pilot	On a bench walkthrough in wk21, told me the abort and confirm buttons in the ground station were dangerously close together; later redesigned the button layout on paper for me	I separated them, added a confirmation dialog, re-tested. Avoided a foreseeable accident at flight day 3 before the hardware even failed
Project manager	Booked farm slots, filed risk assessments, handled transport and insurance paperwork; told me frankly that my docs were “too dense to read”	Protected my time from admin I would otherwise have neglected; her feedback directly triggered the <code>COLLEAGUE_CHECKLIST.md</code> and the one-page quick-ref
Demetro	FDR-prep feedback on how I explained the mission helped me compress the narrative for D6	Got me from a 40-page draft toward a tighter story; feedback loop into §3a of D6
Zian	Picked up mechanical/CAD overflow in wk18–wk20 after joining late, freeing Edward to pair with me on optics instead of fire-fighting hardware alone	Unblocked a week of Edward’s attention on FOV calibration that would not have existed otherwise
Supervisor	Milestone feedback at PDR/IDR/FDR – notably that the project’s “evaluation story” was stronger than its “build story,” which I took as permission to foreground the capability audit	Reframed how I wrote the final deliverables: the 7.7 vs 2.1 scoring is a direct response to that feedback

The non-technical help mattered as much as the technical. The project manager booking field slots while I built code meant that when we arrived at flight day 1 on 12 March we had a trained model, working simulator, progressive test plan, and printed checklists, instead of a half-finished stack and a bag of

paperwork. I did not appreciate that division of labour in wk14; I did by wk20. I now see this is because I had been implicitly equating “contribution” with “lines of code” – a criterion the simulator itself helped dismantle, producing more value per hour than any other artefact without changing any physical world state at all.

4b. What I did that helped others work more effectively. [M16.c][M17.a][M17.b][M17.c] The programme below is past-tense: each item is something I actually did, with a named recipient, a named medium, and a measurable outcome. I order the columns so *medium* is explicit, because the M17 rubric rewards evidence of using a wide range of media.

Recipient	Medium	What I did	Observed behaviour change
Robin (tech)	README + JSON schema + CLI + demo	<code>passive_watch_2.py</code> with <code>--robin</code> flag, <code>cv_mode</code> HTTP polling, JSON schema README [1] [wk20, a5b1ab7 885cee4 36b6069]	Integrated without reading my CV code; zero debugging questions; said the README was “the first docs I had that I could paste directly into my script”
Demetro (tech)	HTML slides + teleprompter sidebar + speaking scripts	2 FDR slides, 246/219-word scripts, polished 4–5× after rehearsal [wk21, 32fb0c5 b2a9cd4 ba667f2 9b826f8]	Delivered a 3.1-min talk; teleprompter sidebar changed his delivery from reading-off-notes to eye-contact
Edward (tech)	Python tool + click-UI + protocol note	Paired on FOV derivation, wrote <code>tools/fov_calibrate_video.py</code> and click-and-label protocol	Reused the tool independently in a later video session; now owns FOV maintenance
Pilot (non-tech)	Browser GUI + large labelled buttons + paper layout redesign	Multiple bench walkthroughs of <code>pi_flight.py</code> ; wider button spacing after his abort-button feedback; Y/N/M/E/W/S legend	Operated ground station unaided on flight day 3 while I diagnosed other failures; translated “state-machine” into “hand-reach vocabulary”
PM (non-tech)	One-page A4 quick-ref + verbal brief	After she said docs were “too dense” I wrote <code>FIELD_QUICK_REF.md</code> and <code>COLLEAGUE_CHECKLIST.md</code> ; walked her through before each trip	Verbally briefed the team without reading code; called the one-pager “the first thing I actually took to the farm on paper”
Whole team (mixed)	Formal letter to course organiser	Team complaint letter, 6 iterations / 4 days, factual tone [26e3255→4b0fd91] [2]	Context on record for assessment; team advocacy by the one person with capacity
Next cohort (external)	Onboarding checklist + pipeline walkthrough	<code>COLLEAGUE_CHECKLIST.md</code> , <code>VISION_PIPELINE_FOR_COLLEAGUE.md</code> wk22	Not yet tested; signal: can a future student get first detection on a Pi in one afternoon without messaging me?

Evaluating the communication methods I used. [M17.d] The methods I reached for were: (1) Markdown prose, (2) code contracts (JSON schema + CLI examples), (3) running executable demos (20-line MAVLink heartbeat, Robin integration script), (4) a browser GUI for the pilot, (5) HTML slides with a teleprompter sidebar for Demetro’s FDR, (6) a formal complaint letter, and (7) face-to-face bench walkthroughs for the pilot and project manager. My criterion for effectiveness, per recipient: did their behaviour change as I intended without a follow-up conversation? Judgement: the browser GUI and the JSON contract scored highest (zero follow-ups); the HTML slides scored high on a narrow target (Demetro’s eye contact improved); Markdown prose scored lowest, because every teammate who read it had follow-ups and the project manager told me it was too dense. The comparison worth recording is that the same underlying content – “how to run the drone detection pipeline” – produced a usable artefact only when I re-cast it as a GUI plus a JSON contract instead of a Markdown document. The lesson: for engineering teammates a runnable contract beats prose; for non-engineering teammates a visual or physical artefact beats both. Default next time: *what is the smallest runnable thing I could hand over?*

The non-technical audience, named explicitly. The pilot and the project manager are the two non-technical audiences on the project – the pilot is a licensed operator, not an engineer; the project manager coordinates logistics and does not read code. For the pilot I replaced keyboard shortcuts with a touch-friendly browser UI, renamed “GUIDED” to “Manual,” and spaced the abort and confirm buttons apart after his direct feedback. For the project manager I collapsed three dense docs into one A4-sized `FIELD_QUICK_REF.md` and walked her through it verbally before each trip. In both cases the feedback loop was direct – they told me what was unclear, I changed it, they used the next version without asking – the adaptation-plus-reception pattern the rubric asks for.

The feedback I gave that changed behaviour. [Insight 70+] Three explicit instances, chosen because I interrogate my own delivery, not just the outcome. (1) I told Demetro, after his first rehearsal, that his slides read as a list of components instead of a story about uncertainty; I suggested he open with the problem (“can we trust the detection?”) and rewrote the scripts accordingly. He adopted it. Delivery was measured – I had watched him rehearse before speaking, and the critique was directed at the artefact, not the person. (2) I told Robin we needed to move from “calling each other’s functions” to “contracts and JSON,” and I wrote the contract. He adopted it. Delivery was less measured – I solved it rather than asked him to – and the better version would have been “here are two options, let’s pick one together.” (3) I told the pilot the ground-station buttons were following my mental model rather than his hand position, and asked him to redesign the layout for me. He did, and I implemented what he drew. That one was the best of the three because I genuinely flipped the authority relationship – pilot as designer, me as draftsman.

My self-development programme, named and falsifiable. [Insight 70+] I am running the following programme, started inside the project timeline and carrying into my next role: (a) one book per month on collaborative engineering – started Schön [5] in wk18, Edmondson [9] in wk21, next is McConnell [8]; (b) a paired-programming session at least once per week in any technical role I hold, with the goal that my pair could take over my work inside one sprint, measured by a dry-run handover; (c) a personal after-action review per engineering sprint, in the `MEMORY.md` format that worked here; (d) a shared *decision log* in every future team (one-liner per decision: chosen, why, rejected), a direct consequence of the three unresolved conflicts in §2c. Falsifiability test: if in six months I cannot point to a teammate who can do a piece of my job without me, the programme has failed.

Closing: what I learned about support that I did not know in January. Help is rarely technical and rarely symmetric. The project manager’s logistics gave me time I spent on code; my code gave her a simulator to demo. I spent most of the project measuring my contribution in commits; the real currency was *did someone else do their job better because of what I did?* On that scale, the Robin contract, Demetro’s scripts, the pilot’s redesigned button layout, and the complaint letter are the contributions I am proudest of – more than the simulator itself. I leave this project believing that engineering leadership is less about being the person who can fix anything and more about making disagreement safe and inheritance easy. The moment this became clear to me was the night I realised I would rather Robin never needed my help again than that he thank me for giving it.

- | | |
|--|--|
| <p>[1] D. Chuprynyuk. <i>Robin Integration Handoff: passive_watch_2.py --robin Protocol</i>. Internal project document, <code>docs/ROBIN_INTEGRATION.md</code>. Commit <code>a5b1ab7</code>, AENGM0074 SAR drone project. 2026.</p> <p>[2] D. Chuprynyuk and AENGM0074 Group. <i>Student Feedback on Hardware Provision and Flight Testing</i>. Internal project document, <code>docs/COMPLAINT_LETTER.md</code>. University of Bristol, AENGM0074. 2026.</p> <p>[3] R. M. Belbin. <i>Team Roles at Work</i>. 2nd. Butterworth-Heinemann, 2010.</p> <p>[4] N. Hatton and D. Smith. “Reflection in Teacher Education: Towards Definition and Implementation”. In: <i>Teaching and Teacher Education</i> 11.1 (1995), pp. 33–49.</p> <p>[5] D. A. Schön. <i>The Reflective Practitioner: How Professionals Think in Action</i>. Basic Books, 1983.</p> <p>[6] B. W. Tuckman. “Developmental Sequence in Small Groups”. In: <i>Psychological Bulletin</i> 63.6 (1965), pp. 384–399.</p> | <p>[7] D. Chuprynyuk. <i>main.py refactor: 1411→754 lines, 6 extracted modules, 146 pytest tests</i>. Project session log, 20 March 2026. <code>memory/session-2026-03-20.md</code>. 2026.</p> <p>[8] S. McConnell. <i>Code Complete</i>. 2nd. Microsoft Press, 2004.</p> <p>[9] A. Edmondson. “Psychological Safety and Learning Behavior in Work Teams”. In: <i>Administrative Science Quarterly</i> 44.2 (1999), pp. 350–383.</p> <p>[10] J. A. Moon. <i>A Handbook of Reflective and Experiential Learning: Theory and Practice</i>. RoutledgeFalmer, 2004.</p> <p>[11] D. Chuprynyuk. <i>Target GPS estimation accuracy progression (7 m → 1.5 m CEP50)</i>. Commits <code>1d8c383</code>, <code>dbd24f3</code>, <code>94144bd</code>, <code>aab8a1f</code>, <code>dc67a8e</code>. AENGM0074 SAR drone project. 2026.</p> <p>[12] N. G. Leveson. <i>Engineering a Safer World: Systems Thinking Applied to Safety</i>. MIT Press, 2011.</p> |
|--|--|